

Trabalho 2 – Sistemas Distribuídos

Realizado por:

Tiago Pinto 54718

João Loios 55469

Introdução

O nosso objetivo com este trabalho é desenvolver um sistema distribuído para monitorizar a temperatura e a humidade de diversas áreas do novo hospital de Évora. O sistema deverá receber dados enviados por dispositivos IoT instalados no edifício, processá-los, armazená-los e disponibilizar funcionalidades para consulta e gestão.

Implementação

Para a construção do trabalho, começámos pela implementação do simulador dos dispositivos IoT. Aqui estabelecemos a ligação ao broker, que é o “mosquitto”, optámos por este broker pela sua simplicidade e porque ambos conseguimos usá-lo sem problema. Através desse broker mandamos mensagens do tipo: ID, temperatura, humidade, timestamp.

Criamos 6 dispositivos que enviam para o server 1 mensagem deste tipo a cada 15 segundos.

Seguidamente implementámos a classe principal do nosso trabalho, a classe “Server”, esta classe é responsável por algumas funções importantes:

- Primeiramente acedemos ao ficheiro `parametros.properties` que contém a maioria dos parâmetros de configuração.
- Em segundo estabelecemos a conexão à base de dados, optamos por implementar de forma similar ao primeiro trabalho, pois não conseguimos implementar com JPA até à data limite. Temos duas tabelas na base de dados, uma “devices” e outra “metrics” na devices temos as colunas ID->character(2) pois assumimos que não haverá mais de 99 dispositivos simultâneos, edificio->character(1) pelos mesmos motivos, piso->character(2) pelos mesmos motivos, sala->character(3), porque assumimos que não haverá mais de 99 salas com outro caracter necessário, ex:12A, servico->character(50), pelos mesmos motivos. Nota-se que todos os tipos são characters o que pode parecer contra-intuitivo, mas como não é necessário realizar operações ou relações matemáticas com estes valores, e como foi mais útil para nós na execução do programa, queremos denotar que esta foi uma decisão consciente. Esta tabela tem como chave-primária, obviamente o ID. Já a

tabela metrics tem 4 colunas, o ID, chave-estrangeira da tabela devices, temperature->numeric, humidity->numeric e timestamp->timestamp with time zone NOT NULL esta tabela tem como chave primaria o ID e o timestamp, pois como o timestamp tem os dados até aos segundos assumimos que o mesmo dispositivo não consegue fazer 2 métricas no mesmo segundo.

- De seguida inserimos na base de dados 5 devices e 4 metrics, estabelecem então a conexão ao broker que ao receber as tais 6 mensagens por cada 15 segundos verifica se o ID existe na tabela devices e consoante essa informação descarta ou insere na tabela a métrica.
- Implementamos então a REST API usando JAX-RS devido á sua simplicidade e fácil compreensão, iniciamos o Httpserver usando o Grizzly pois é bem compatível com o JAX-RS, criamos também diversos endpoints sendo eles, AddDevice->/devices, que adiciona um novo dispositivo á base de dados, listDevices->/devices, que lista todos os dispositivos, updateDevice->/devices/{id} que atualiza uma coluna do dispositivo com o id correspondente, deleteDevice->/devices/{id} que remove o dispositivo com o id correspondente, averageTemperature->/devices/averageTemperature/{filtros} que retorna a média das temperaturas correspondentes aos filtros aplicados, averageHumidity->/averageHumidity/{filtros} que faz o mesmo da anterior mas com a humidade.

De seguida, implementamos a classe “Client_Admin” que serve para interagir com a API REST, permitindo este consultar as temperaturas e humidades de qualquer horário em qualquer local do hospital via menu e gerenciar os dispositivos via linha de comandos.

Por fim, implementamos a classe “Autorizacao” que é responsável por verificar se o cliente que está a tentar aceder ao servidor apresenta as credenciais válidas no cabeçalho de autorização.

Instruções de utilização

De modo a utilizar este serviço pode fazê-lo de 2 maneiras diferentes:

- Pode utilizar o script shell para correr o trabalho todo, mas antes de fazê-lo terá de alterar o caminho “cd /home/tiago/sd-t02-54718-55469”, deforma a ser compatível com o seu dispositivo.
- Correr os 3 ficheiros principais que compõem o trabalho (utilizando o “run” do java) na seguinte ordem: Server.java -> IoTDevicePublisher.java -> Client_admin.java.

Conclusão

Ao realizar este trabalho, tivemos mais dificuldades ao utilizar o JPA para a construção automática das tabelas no arranque da aplicação, por isso como solução, decidimos trabalhar com as tabelas de igual forma ao trabalho passado. Também tivemos algumas dificuldades ao implementar o API REST, mas após algum esforço conseguimos sucesso. Por fim, apesar de desafiador, gostamos de realizar este trabalho, e pretendemos melhorar os nossos conhecimentos e adquirir mais, futuramente.